LEARN BEING FORWARD

Programming — C++ Programming

STL — containers & algorithms

COMPLETE LEARNING GUIDE

What You'll Learn

The Standard Template Library (STL) is the single biggest productivity multiplier in modern C++: a battle-tested, highly optimized set of generic containers (vector, map, set, and more) and algorithms (sort, find, transform) that eliminate the need to hand-roll data structures the way C programmers routinely must. Knowing the STL well is as much about judgment as syntax — picking vector vs. list vs. deque, or map vs. unordered_map, has real, sometimes dramatic performance consequences depending on the access pattern a program actually needs. This sub-module covers the core sequence and associative containers, iterators and the invalidation rules that trip up even experienced C++ developers, the algorithms library's composable function-based design, lambda expressions as the modern way to supply custom logic to an algorithm, and practical guidance for choosing the right container.

1. Sequence Containers: vector, deque, and list

std::vector is a dynamically resizable array stored contiguously in memory, giving O(1) random access by index and excellent cache performance, but inserting or removing anywhere except the end is O(n) because every following element must shift. std::deque (double-ended queue) supports O(1) insertion/removal at both ends (unlike vector, which is only efficient at the end) at the cost of non-contiguous storage internally. std::list is a doubly-linked list giving O(1) insertion/removal anywhere once you have an iterator to the position, but O(n) random access and worse cache locality than vector — in practice, vector is the correct default choice unless you have a specific, measured reason to reach for list or deque.

```cpp
#include <vector>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 2, 3};
    nums.push_back(4);      // O(1) amortized — append at the end
    nums.insert(nums.begin(), 0); // O(n) — shifts every element right

    for (int n : nums) std::cout << n << " "; // 0 1 2 3 4
    std::cout << "\n";
    return 0;
}
```

2. Associative Containers: map, set, and unordered_map

std::map stores unique keys in sorted order (by default, ascending) with O(log n) lookup/insert/erase, implemented internally as a balanced binary search tree — iterating a map always visits keys in sorted order. std::unordered_map trades that ordering guarantee for average O(1) lookup/insert/erase via hashing, making it the right default when insertion order and key ordering don't matter and raw speed does. std::set and std::unordered_set are the equivalent structures for storing unique values with no associated mapped value, used whenever you need fast membership testing ('have I seen this value before?') without duplicates.

```cpp
#include <unordered_map>
#include <iostream>

int main() {
    std::unordered_map<std::string, int> ages;
    ages["Asha"] = 30;
    ages["Ravi"] = 25;

    if (ages.find("Asha") != ages.end()) {
        std::cout << "Asha is " << ages["Asha"] << "\n"; // Asha is 30
    }

    for (const auto &pair : ages) { // order is unspecified for unordered_map
        std::cout << pair.first << ": " << pair.second << "\n";
    }
    return 0;
}
```

3. Iterators and Iterator Invalidation

An iterator is a generic 'pointer-like' object that can traverse any container uniformly — begin() returns an iterator to the first element, end() returns a sentinel one-past-the-last-element iterator, and ++it advances it — the same interface works whether the underlying container is a vector, a list, or a map. The critical gotcha is iterator invalidation: operations that resize or reallocate a container's internal storage (vector's push_back when capacity is exceeded, or erase on most containers) can invalidate some or all existing iterators, pointers, and references into that container — continuing to use an invalidated iterator is undefined behavior, and erase's own return value (a valid iterator to the next element) is the standard, safe way to keep iterating after removing an element.

- After any vector reallocation (triggered by push_back exceeding current capacity), every previously held iterator, pointer, and reference into that vector is invalidated.
- it = container.erase(it); (reassigning from erase's return value) is the correct pattern for removing elements while iterating — incrementing an iterator just erased is undefined behavior.

```cpp
#include <vector>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 2, 3, 4, 5};

    // Remove all even numbers, using erase's return value to stay valid
    for (auto it = nums.begin(); it != nums.end(); ) {
        if (*it % 2 == 0) {
            it = nums.erase(it); // erase returns the next valid iterator
        } else {
            ++it;
        }
    }

    for (int n : nums) std::cout << n << " "; // 1 3 5
    std::cout << "\n";
    return 0;
}
```

4. The Algorithms Library

<algorithm> provides generic, container-agnostic functions that operate on a range specified by a pair of iterators (begin, end) — std::sort sorts a range in place, std::find searches linearly for a value and returns an iterator to it (or end() if not found), std::transform applies a function to every element and writes the results to a destination, and std::accumulate (in <numeric>) folds a range down to a single value (like a running sum). Because these algorithms work on iterator ranges rather than specific container types, the exact same std::sort call works identically on a vector, a deque, or a plain array — this uniformity is the core design principle of the STL.

```cpp
#include <vector>
#include <algorithm>
#include <numeric>
#include <iostream>

int main() {
    std::vector<int> nums = {5, 3, 1, 4, 2};

    std::sort(nums.begin(), nums.end()); // 1 2 3 4 5

    auto it = std::find(nums.begin(), nums.end(), 3);
    if (it != nums.end()) std::cout << "Found 3 at index " << (it - nums.begin()) << "\n";
    int total = std::accumulate(nums.begin(), nums.end(), 0); // sum = 15
    std::cout << "Sum = " << total << "\n";
    return 0;
}
```

5. Lambda Expressions as Predicates and Comparators

A lambda expression ([capture](parameters) { body }) defines an unnamed, inline function object, and is the standard modern way to supply custom comparison or filtering logic to an algorithm without writing a separate named function or functor class. The capture clause controls which outer variables the lambda can access and how ([x] captures x by value/copy, [&x] captures it by reference, [&] captures everything used by reference, [=] captures everything used by value) — getting capture-by-reference wrong when the lambda outlives the captured variable is a common source of dangling-reference bugs, exactly analogous to returning a reference to a local variable.

```cpp
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    std::vector<int> nums = {5, 3, 1, 4, 2};

    // Sort descending using a lambda comparator
    std::sort(nums.begin(), nums.end(), [](int a, int b) { return a > b; });
    for (int n : nums) std::cout << n << " "; // 5 4 3 2 1
    std::cout << "\n";

    int threshold = 3;
    auto it = std::find_if(nums.begin(), nums.end(),
                           [threshold](int n) { return n < threshold; }); // captures by value
    if (it != nums.end()) std::cout << "First below threshold: " << *it << "\n";
    return 0;
}
```

6. Choosing the Right Container

The practical default is std::vector unless a specific access pattern justifies otherwise: need fast lookup by a key, don't care about order !’ unordered_map/unordered_set; need sorted iteration order !’ map/set; need fast insertion/removal at both ends !’ deque; need fast insertion/removal in the MIDDLE with stable iterators to other elements !’ list (rare in practice). Measuring with real data and a profiler beats guessing — vector is cache-friendly enough that it frequently outperforms list even for insertions in the middle, for all but fairly large element counts, because of how much contiguous memory access benefits modern CPU caches.

- Default to vector; only reach for another container once you have a specific, measured performance reason.
- map/set guarantee sorted iteration order at O(log n) operations; unordered_map/unordered_set trade that ordering away for average O(1) operations.

7. Range-Based for Loops and auto

The range-based for loop (for (auto &elem : container)) iterates any container that provides begin()/end() without manually managing an iterator at all, and is the idiomatic default for simple traversal when the index itself isn't needed. Using auto for the element type avoids both verbose type names (std::unordered_map<std::string, std::vector<int>>::iterator) and subtle copy bugs — for (auto elem : container) copies each element, while for (auto &elem : container) binds a reference and avoids the copy (use const auto & when you don't need to modify elements, to additionally prevent accidental mutation).

```cpp
#include <vector>
#include <string>
#include <iostream>

int main() {
    std::vector<std::string> names = {"Asha", "Ravi", "Meera"};

    for (const auto &name : names) { // reference avoids copying each string
        std::cout << name << "\n";
    }
    return 0;
}
```

Common Pitfalls

- Continuing to use a vector iterator, pointer, or reference after an operation (push_back past capacity, erase, insert) that invalidated it.
- Incrementing an iterator just passed to erase() instead of reassigning from erase's return value.
- Defaulting to std::list for 'fast insertion' without measuring — std::vector's cache locality frequently wins even for middle insertions at realistic sizes.
- Using std::map when insertion-order or sorted-order isn't actually needed, paying O(log n) instead of unordered_map's average O(1).
- Capturing a local variable by reference in a lambda that outlives the variable's scope, producing a dangling reference when the lambda is later invoked.
- Calling container[key] on a map/unordered_map just to check existence — operator[] silently inserts a default-constructed entry for a missing key.

Key Takeaways

- std::vector is the correct default container; reach for deque, list, map, or unordered_map only for a specific, justified access pattern.
- Operations that resize/reallocate a container can invalidate existing iterators, pointers, and references into it — know which operations do this for your chosen container.

- map/set guarantee sorted order at O(log n); unordered_map/unordered_set trade ordering for average O(1) operations.
- Algorithms in <algorithm> operate on iterator ranges, making the same function work uniformly across different container types.
- Lambda capture-by-reference must never outlive the captured variable — the same dangling-reference risk as returning a reference to a local.

Learn More

cppreference — Containers library
https://en.cppreference.com/w/cpp/container

cppreference — Algorithms library
https://en.cppreference.com/w/cpp/algorithm